

Chittagong University of Engineering and Technology (CUET)

Department of Computer Science and Engineering

S/W Development Project (Sessional) | Course Code: CSE 300

Project Proposal Report

BringIt- Peer Delivery System

Submitted By

Name	Student ID
Md. Abdulla Al Nishat	2204042
Mohammad Sakib Hasan	2204051
Ayan Barua	2204053
Fatema Jannat Rup	2204064

Submitted To

Sharmistha Chanda Tista

Assistant Professor, Department of CSE, CUET

Md. Atiqul Islam Rizvi

Assistant Professor, Department of CSE, CUET

Date of Submission: August 8, 2026

August 6, 2026

Contents

1 Introduction **2**

2 Background Study **2**

3 Motivation **2**

4 Application **3**

5 Proposed Methodology **3**
5.1 Delivery Request Status Flow 4

6 Features **5**

7 Tools **6**

8 Conclusion **6**

1 Introduction

Campus life frequently involves small, recurring logistical tasks – picking up food from the canteen, delivering documents between departments, forwarding library books, or getting items across a large campus – that are inconvenient for a single student to interrupt their schedule for, yet too minor to justify a formal courier service. **BringIT** is a peer-to-peer (P2P) delivery request platform designed specifically for a university campus environment. It connects students who need something delivered (*requesters*) with fellow students who are willing to carry it out along their existing route (*deliverers*), turning everyday campus movement into a mutually beneficial, community-driven delivery network.

The platform is built as a full-stack web application using React for the frontend and Supabase (PostgreSQL, Authentication, Realtime, and Storage) for the backend, allowing users to authenticate securely, create and track delivery requests, communicate in real time, and build reputation through a gamified leaderboard system.

2 Background Study

Commercial delivery and courier platforms such as Pathao, Uber, and Foodpanda solve general-purpose delivery problems but are not tailored to the scale, cost sensitivity, or trust structure of a university campus. Existing solutions have several shortcomings when applied to a campus context:

- **Cost inefficiency**: Commercial services charge delivery fees disproportionate to the short distances typical of a campus (e.g., one building to another).
- **Lack of trust and community**: Deliveries are handled by unknown third-party riders rather than verified peers from the same institution.
- **No academic-community integration**: General platforms have no concept of student identity, class schedules, or campus geography, and therefore cannot optimize for "on-the-way" deliveries.
- **Manual, informal coordination**: In practice, students currently coordinate small favors through scattered WhatsApp/Facebook groups or word of mouth, with no tracking, accountability, or incentive structure.

This gap motivates the study of a purpose-built, campus-scoped delivery coordination system that leverages student identity verification, real-time communication, and lightweight gamification to build a reliable peer delivery network.

3 Motivation

The motivation for BringIT stems from recurring, first-hand observations of campus life:

1. **Time scarcity**: Students often cannot leave a lab, class, or hall to pick up a parcel, food order, or document, even though another student is already walking that exact route.
2. **Underutilized peer trust**: Students within the same institution already share an implicit layer of trust (verified by student ID) that commercial platforms cannot replicate.

3. **No incentive for informal help**: Existing informal delivery favors are unrewarded and unstructured, discouraging repeated participation.
4. **Opportunity for a lightweight, low-cost solution**: A web application requiring no proprietary hardware, built entirely on modern, free-tier-friendly technologies (React, Supabase), can be deployed and iterated on quickly for a student-sized user base.

These observations directly motivate a system that formalizes peer delivery requests, adds accountability through tracking and authentication, and rewards active deliverers – ultimately increasing convenience for requesters and creating a rewarding, community-oriented side activity for deliverers.

4 Application

BringIT is intended for deployment within a single university or a cluster of geographically close campuses. Concrete use cases include:

- A student stuck in a lab session requests that a canteen order be picked up and dropped at their department.
- A student delivers a lab report or physical document from one faculty building to another on their way to class.
- A resident of a dormitory requests that a parcel be collected from the campus post office / gate.
- Frequent deliverers build a public reputation via the leaderboard, which can later be tied to campus recognition or micro-incentive programs run by student bodies.

Beyond individual convenience, the platform has secondary value as a community-engagement tool: it encourages students to interact across departments and years, and generates a dataset of campus mobility and demand patterns that could inform future improvements such as route optimization or partnership with campus administration.

5 Proposed Methodology

Figure 1 outlines the six-stage methodology used to design and build BringIT:

1. **Requirement Gathering & Problem Analysis**: Campus delivery pain points, user roles (requester/deliverer), and core functional needs were identified before any design work began.
2. **System Design**: The overall architecture, database schema, and UI/UX flow were planned, laying the foundation for both frontend and backend work.
3. **Parallel Development**: Once the design was finalized, work split into two concurrent tracks – *Frontend Development* (React + Tailwind CSS pages, components, and contexts) and the *Backend & Data Layer* (Supabase authentication, PostgreSQL, Realtime, and Storage) – allowing both halves of the stack to progress independently against the shared design.

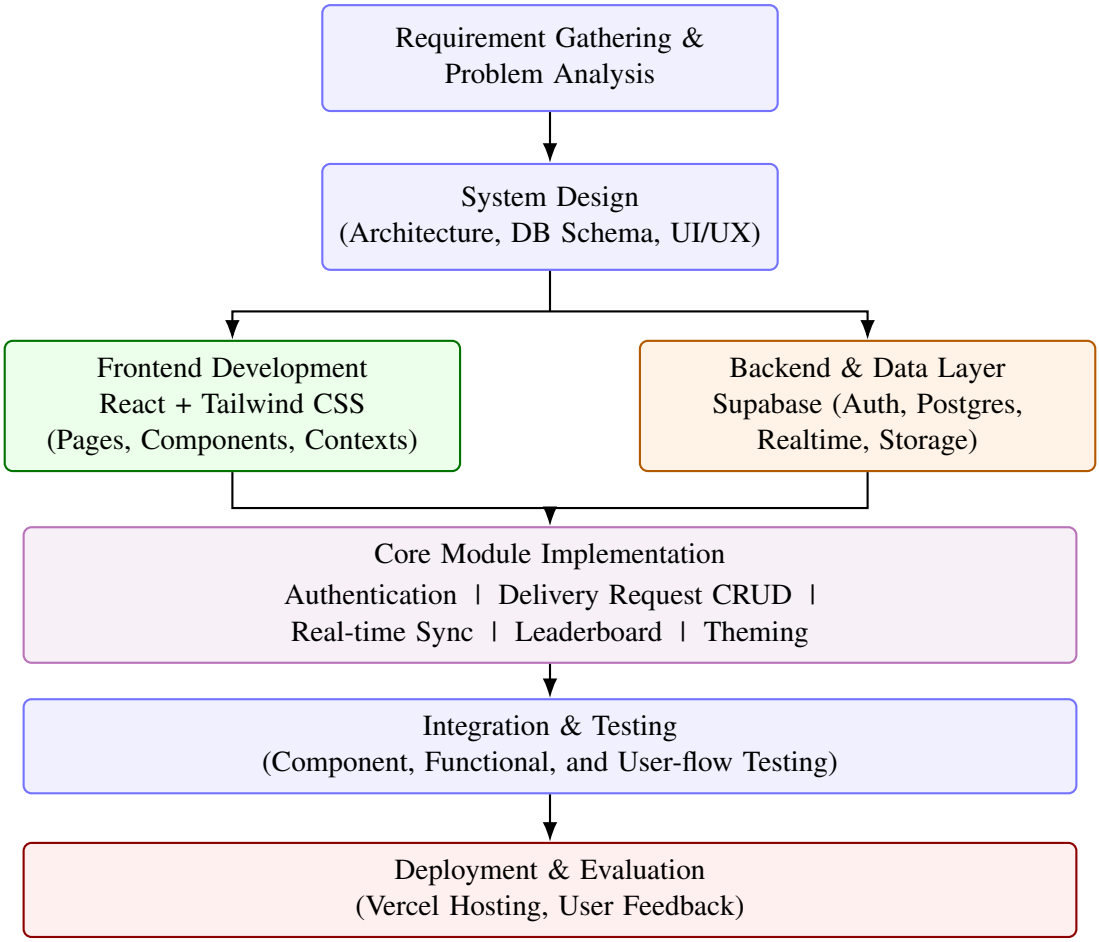


Figure 1: Development methodology and workflow followed for BringIT

4. **Core Module Implementation**: The two tracks converge here, where the actual feature modules (authentication, delivery request CRUD, real-time synchronization, leaderboard, and theming) are implemented by wiring the frontend to the backend.
5. **Integration & Testing**: The completed modules are integrated and verified through component-level, functional, and end-to-end user-flow testing to catch issues before release.
6. **Deployment & Evaluation**: The application is deployed (via Vercel) and evaluated using real user feedback, closing the loop back into future iterations of the design.

This is essentially an **Agile, iterative** methodology: rather than a strict one-way waterfall, the boundary between core implementation, testing, and deployment is revisited in short cycles as feedback comes in, allowing features such as route-based matching or push notifications to be layered on in later iterations without redoing the whole pipeline.

5.1 Delivery Request Status Flow

Figure 2 depicts the status flow of a delivery request from creation to completion.

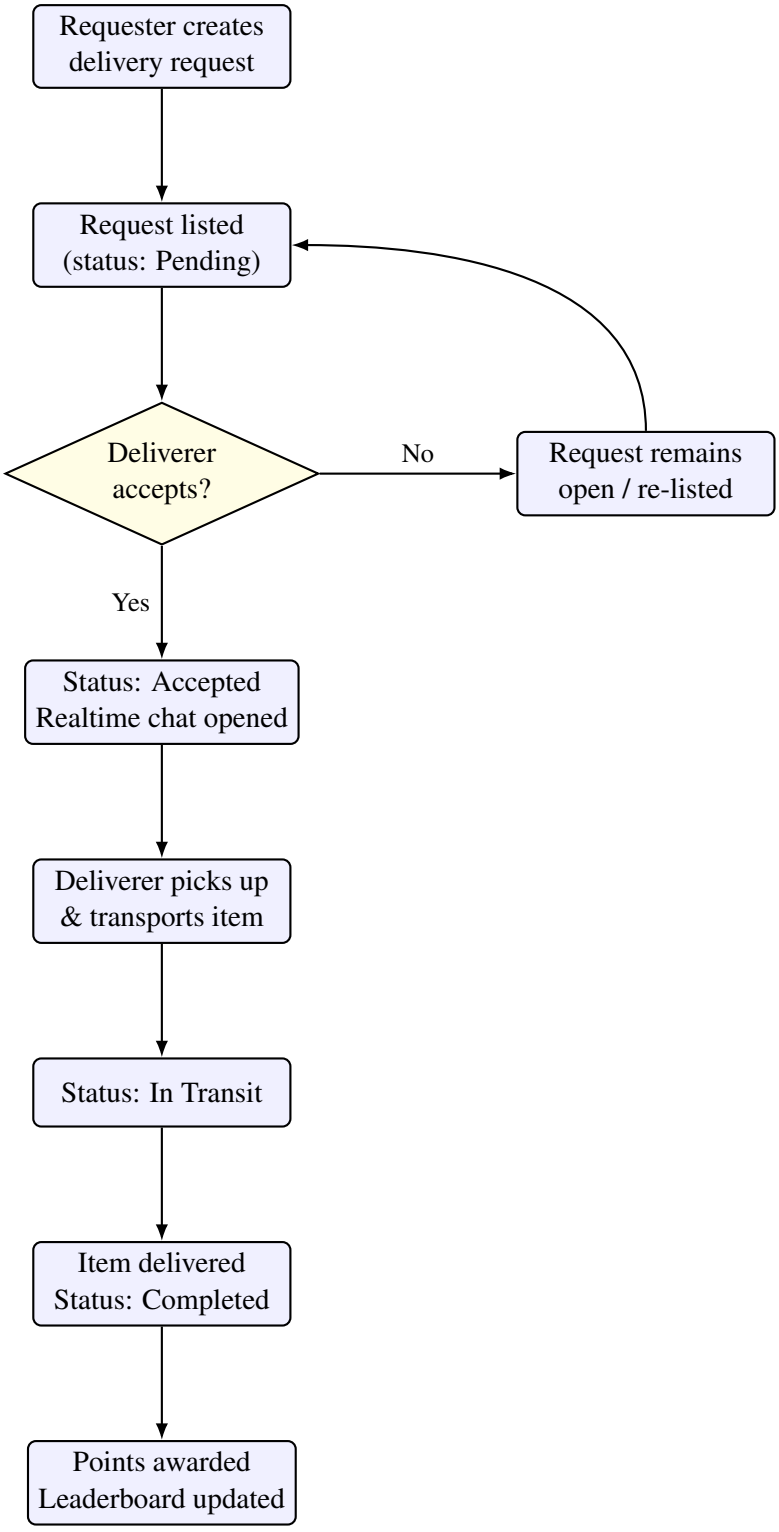


Figure 2: Delivery request lifecycle and status flow

6 Features

- **Secure Authentication & Profiles**: JWT-based sign-up/login via Supabase Auth; users must complete a profile (including `student_id`) before creating or accepting requests.
- **Delivery Request Management**: Full CRUD support for delivery requests, including

detailed nested views for tracking individual request status and history.

- **Real-Time Synchronization**: WebSocket-based `useRealtime` hooks provide live status updates, in-app notifications, and instant messaging between requester and deliverer.
- **Leaderboard Gamification**: A point-based leaderboard rewards frequent, reliable deliverers and encourages sustained platform engagement.
- **Customizable UI/UX**: Dark/Light theme switching, animated components (Framer Motion-style interactions), toast notifications, and context-driven modal dialogs.
- **Responsive, Mobile-First Design**: A fully responsive layout built with Tailwind CSS, ensuring usability across phones, tablets, and desktops – important given most students will access the platform on mobile devices while on campus.

7 Tools

Category	Tools / Technologies
Frontend Framework	React 19, React Router v7, Vite (bundler)
Styling	Tailwind CSS v4, `clsx`, `tailwind-merge`
Backend / Database	Supabase (PostgreSQL, Authentication, Realtime, Storage)
Icons & UI Assets	Lucide React
Version Control & Hosting	Git, GitHub, Vercel (deployment)
Development Environment	Visual Studio Code, Node.js (v18+), npm

Table 1: Development tools and technology stack

8 Conclusion

BringIT addresses a practical, everyday problem faced by university students by introducing a structured, trust-based, peer-to-peer delivery platform tailored to campus life. By combining a modern React frontend with the Supabase backend-as-a-service ecosystem, the project achieves rapid development velocity while still delivering core requirements: secure authentication, real-time request tracking, in-app communication, and gamified incentives. The proposed Agile methodology and modular architecture ensure the system can be extended incrementally – future iterations may include route-based matching, push notifications, and analytics on campus delivery patterns. Overall, BringIT is expected to demonstrate how lightweight, community-driven software can meaningfully improve convenience and collaboration within a university ecosystem.